

Statistical Programming II

Practical 3: Debugging

Exercise 1–5 答案与复习笔记

目录

第一部分 中文答案	3
1 Exercise 1: 调试带列参数的函数	3
1.1 问题代码	3
1.2 修正答案	3
1.3 本题要点	4
2 Exercise 2: 错误发生在辅助函数内部	4
2.1 问题代码	4
2.2 错误原因	4
2.3 修正答案	5
2.4 本题要点	5
3 Exercise 3: 使用 browser() 调试递归函数	5
3.1 原始函数及递归逻辑	5
3.2 加入 browser()	6
3.3 为什么不能只输入 n	6
3.4 my_factorial(0) 为什么无法结束	6
3.5 my_factorial(3.5) 为什么也不行	7
3.6 常用调试命令	7
3.7 如何理解 where	7
3.8 正确修复	8
4 Exercise 4: 使用 debug() 和 debugonce()	8
4.1 问题函数	8
4.2 使用 debugonce()	9
4.3 修正答案	9
4.4 debug() 与 debugonce() 的区别	10

5 Exercise 5: 调试 Quarto 文档	10
5.1 调试思路	10
5.2 错误一: 用 = 代替 ==	11
5.3 错误二: 没有加载 ggplot2	11
5.4 错误三: echo=TREU	11
5.5 完整修复后的 broken.qmd	12
5.6 判断错误属于哪一层	13
5.7 HTML 与 PDF 渲染的差别	13
 第二部分 Deutsche Fassung	 14
6 Exercise 1: Spaltenargumente mit tidy evaluation	14
6.1 Problem	14
6.2 Korrektur	14
 7 Exercise 2: Fehler in einer Hilfsfunktion	 14
7.1 Fehlerursache	14
7.2 Korrektur	15
 8 Exercise 3: Rekursion mit browser() untersuchen	 15
8.1 Warum die Funktion fehlschlägt	15
8.2 Interaktives Debugging	15
8.3 Korrektur	16
 9 Exercise 4: debug() und debugonce()	 16
9.1 Fehleranalyse	16
9.2 Korrektur	17
 10 Exercise 5: Ein Quarto-Dokument debuggen	 17
10.1 Die drei Fehler	17
10.2 Korrigiertes Dokument	18
10.3 Fehlerebenen	18
 Kurzfazit	 19

第一部分 中文答案

1 Exercise 1: 调试带列参数的函数

1.1 问题代码

```
1 mean_mass_by <- function(data, group_var) {  
2   data |>  
3     group_by(group_var) |>  
4     summarise(  
5       mean_mass = mean(body_mass_g, na.rm = TRUE)  
6     )  
7 }
```

如果调用：

```
1 mean_mass_by(penguins, species)
```

函数中的 `group_by(group_var)` 不会自动把 `group_var` 替换为调用时传入的列 `species`。`dplyr` 的 `group_by()` 使用 data masking（数据掩码）和 tidy evaluation（整洁求值）。在自定义函数中转交一个未加引号的列名时，需要明确告诉 `dplyr`：这里传入的是一列表达式。

1.2 修正答案

```
1 mean_mass_by <- function(data, group_var) {  
2   data |>  
3     group_by({{ group_var }}) |>  
4     summarise(  
5       mean_mass = mean(body_mass_g, na.rm = TRUE),  
6       .groups = "drop"  
7     )  
8 }  
9  
10 mean_mass_by(penguins, species)
```

`{{ group_var }}` 称为 embracing。可以把它理解为：

取得调用者传入的列名表达式，并把它放到 `group_by()` 中求值。

这里传参时不需要加引号：

```
1 mean_mass_by(penguins, species)
```

如果函数设计为接收字符串，例如 "species"，则应写成：

```
1 mean_mass_by_string <- function(data, group_var) {  
2   data |>  
3     group_by(.data[[group_var]]) |>
```

```
4     summarise(  
5       mean_mass = mean(body_mass_g, na.rm = TRUE),  
6       .groups = "drop"  
7     )  
8 }  
9  
10 mean_mass_by_string(penguins, "species")
```

1.3 本题要点

- 未加引号的列名参数：使用 `{{ column }}`。
- 字符串形式的列名：使用 `.data[[column]]`。
- `na.rm = TRUE` 用于在计算均值时忽略缺失值。
- `.groups = "drop"` 让结果不再保留分组结构。

2 Exercise 2: 错误发生在辅助函数内部

2.1 问题代码

```
1 mean_body_mass <- function(x) {  
2   mean(x, na.rm = TREU)  
3 }  
4  
5 summarise_species <- function(data) {  
6   data |>  
7     group_by(species) |>  
8     summarise(  
9       mean_mass = mean_body_mass(body_mass_g)  
10    )  
11 }  
12  
13 summarise_species(penguins)
```

运行后会出现类似错误：

```
Error in 'summarise()':  
...  
Caused by error in 'mean()':  
! object 'TREU' not found
```

2.2 错误原因

TREU 是拼写错误。R 的逻辑值必须写成：

```
1 TRUE
2 FALSE
```

R 会把没有引号的 TREU 当作对象名查找；当前环境中不存在这个对象，因此产生 object 'TREU' not found 的错误。

外层报错位置可能显示在 summarise() 中，但真正出错的调用链是：

```
summarise_species()
-> summarise()
  -> mean_body_mass()
    -> mean(..., na.rm = TREU)
```

因此，阅读嵌套错误时不能只看最外层函数，还应继续阅读 Caused by error in ... 部分。

2.3 修正答案

```
1 mean_body_mass <- function(x) {
2   mean(x, na.rm = TRUE)
3 }
4
5 summarise_species <- function(data) {
6   data |>
7     group_by(species) |>
8     summarise(
9       mean_mass = mean_body_mass(body_mass_g),
10      .groups = "drop"
11    )
12 }
13
14 summarise_species(penguins)
```

2.4 本题要点

- 报错显示在外层函数，不等于错误一定发生在外层函数。
- 应沿着 error message 中的调用关系寻找最内层原因。
- TRUE 和 FALSE 是 R 的保留逻辑值，不能随意拼写。
- 将重复计算封装为辅助函数没有问题，但调试时也必须检查辅助函数。

3 Exercise 3: 使用 browser() 调试递归函数

3.1 原始函数及递归逻辑

```
1 my_factorial <- function(n) {  
2   if (n == 1) return(1)  
3   return(n * my_factorial(n - 1))  
4 }
```

对于 `my_factorial(5)`，递归过程是：

```
my_factorial(5)  
= 5 * my_factorial(4)  
= 5 * 4 * my_factorial(3)  
= 5 * 4 * 3 * my_factorial(2)  
= 5 * 4 * 3 * 2 * my_factorial(1)  
= 5 * 4 * 3 * 2 * 1  
= 120
```

`if (n == 1) return(1)` 是 base case（终止条件）。如果递归过程永远无法到达这个条件，函数就无法正常结束。

3.2 加入 `browser()`

```
1 my_factorial <- function(n) {  
2   browser()  
3  
4   if (n == 1) return(1)  
5   return(n * my_factorial(n - 1))  
6 }  
7  
8 my_factorial(0)
```

函数运行到 `browser()` 时会暂停，并显示：

```
Browse[1]>
```

此时函数并没有结束。R 只是把执行暂停，让我们检查当前环境、逐行运行代码，或者查看函数调用链。

3.3 为什么不能只输入 `n`

函数参数叫 `n`，但在 debugger 中，单独的 `n` 也是 `next` 命令。因此：

```
1 print(n) # inspect the current value of n  
2 n        # execute the debugger's "next" command
```

这解释了为什么输入 `n` 后没有打印变量，反而看到了下一行代码。

3.4 `my_factorial(0)` 为什么无法结束

第一次暂停时：

```
Browse[1]> print(n)
[1] 0
```

因为 $0 == 1$ 为 `FALSE`，函数继续调用：

```
1 my_factorial(n - 1)
2 # my_factorial(-1)
```

之后参数按照下面的顺序变化：

```
0 -> -1 -> -2 -> -3 -> -4 -> ...
```

它永远不会等于 1，所以递归无法停止，最终通常会产生栈空间相关错误。问题不在于数学上没有 $0!$ 。事实上：

$$0! = 1.$$

真正的问题是原函数没有把 0 纳入终止条件。

3.5 my_factorial(3.5) 为什么也不行

```
3.5 -> 2.5 -> 1.5 -> 0.5 -> -0.5 -> ...
```

这一序列同样不会准确到达 1。此外，本题中的阶乘函数只应接受非负整数，所以小数应在递归开始前直接被拒绝。

3.6 常用调试命令

命令	含义
<code>print(n)</code>	查看变量 <code>n</code> 的当前值。
<code>n</code>	执行下一条语句；通常不主动进入普通函数内部。
<code>s</code>	<code>step into</code> ：执行下一步，并尝试进入该行调用的函数。
<code>f</code>	<code>finish</code> ：尝试执行完当前函数，然后返回上一层。
<code>c</code>	<code>continue</code> ：继续运行到下一个断点、错误或程序结束。
<code>where</code>	显示当前调用栈。
<code>Q</code>	立即退出当前调试过程。

3.7 如何理解 where

例如：

```
where 1 at #4: my_factorial(n - 1)
where 2 at #4: my_factorial(n - 1)
where 3 at #4: my_factorial(n - 1)
```

```
where 4 at #4: my_factorial(n - 1)
where 5: my_factorial(0)
```

可以简化为:

```
my_factorial(0)
-> my_factorial(-1)
  -> my_factorial(-2)
    -> my_factorial(-3)
      -> my_factorial(-4) # current frame
```

where 1 是最靠近当前执行位置的调用，最后的 where 5 是最早的调用。外层函数都在等待更内层函数返回结果。

Browse[1] 中的数字 1 不是变量 n 的值，也不是当前递归深度。递归层数应通过 where 判断。

3.8 正确修复

```
1 my_factorial <- function(n) {
2   stopifnot(
3     n >= 0,
4     n == as.integer(n)
5   )
6
7   if (n <= 1) return(1)
8
9   return(n * my_factorial(n - 1))
10 }
11
12 my_factorial(0)    # 1
13 my_factorial(1)    # 1
14 my_factorial(5)    # 120
15 my_factorial(3.5)  # Error
```

修复包含两部分:

1. stopifnot() 检查 n 是否为非负整数;
2. n <= 1 同时正确处理 0! 和 1!。

输入检查必须放在终止条件之前，否则负数也可能因为 n <= 1 被错误地返回 1。调试完成后应删除函数中的 browser()。

4 Exercise 4: 使用 debug() 和 debugonce()

4.1 问题函数


```

1 standardise <- function(x) {
2   (x - mean(x)) / sd(x)
3 }
4
5 standardise(c(1, 2, NA, 4, 5))
6 # NA NA NA NA NA

```

标准化公式为：

$$z_i = \frac{x_i - \bar{x}}{s}.$$

只有一个元素为 NA，但结果全部变成了 NA。

4.2 使用 debugonce()

```

1 debugonce(standardise)
2 standardise(c(1, 2, NA, 4, 5))

```

debugonce(standardise) 不会立即运行函数。它表示：

下一次调用 standardise() 时进入调试器；完成这一次调用后，自动取消调试标记。

进入 Browse[1]> 后检查：

```

1 print(x)
2 # [1] 1 2 NA 4 5
3
4 mean(x)
5 # [1] NA
6
7 sd(x)
8 # [1] NA

```

因此原计算实际上变成：

```

1 (x - NA) / NA

```

缺失值参与算术运算时会继续传播，所以所有结果都成为 NA。这称为 NA propagation。

4.3 修正答案

```

1 standardise <- function(x) {
2   (x - mean(x, na.rm = TRUE)) /
3   sd(x, na.rm = TRUE)
4 }
5
6 standardise(c(1, 2, NA, 4, 5))
7 # -1.095445 -0.547723 NA 0.547723 1.095445

```

计算均值和标准差时使用的是：

```
1, 2, 4, 5
```

它们的均值为 3。原本缺失的第三个元素仍然是 NA。 `na.rm = TRUE` 只是让 `mean()` 和 `sd()` 在计算时忽略缺失值，并不会自动填补原数据。

4.4 `debug()` 与 `debugonce()` 的区别

```
1 debug(standardise)
```

使用 `debug()` 后，每次调用函数都会进入调试器，直到手动取消：

```
1 undebug(standardise)
```

也可以检查函数当前是否带有调试标记：

```
1 isdebugged(standardise)
2 # TRUE or FALSE
```

方法	设置位置	生效范围	取消方法
<code>browser()</code>	写在函数内部	每次执行到该行	从函数中删除 <code>browser()</code>
<code>debugonce(f)</code>	函数外部	只调试下一次调用	自动取消
<code>debug(f)</code>	函数外部	调试之后的每次调用	<code>undebug(f)</code>

5 Exercise 5: 调试 Quarto 文档

5.1 调试思路

Quarto 文档从上到下执行代码块。某个 chunk 失败以后，后面的 chunk 通常不会继续执行。因此应采用以下过程：

1. 找到报错信息中第一个真正的错误；
2. 确认出错文件、行号和 chunk；
3. 修复当前错误；
4. 重新渲染，让下一个错误显现；
5. 重复以上步骤，直到渲染成功。

5.2 错误一：用 = 代替 ==

错误代码：

```
1 penguins |>
2   filter(species = "Adelie")
```

= 在函数调用中用于给参数命名，不表示比较。筛选 species 等于 "Adelie" 的行应写成：

```
1 penguins |>
2   filter(species == "Adelie")
```

```
species = "Adelie" # assignment/naming
species == "Adelie" # equality comparison
```

这是 R/dplyr 代码执行阶段的错误。

5.3 错误二：没有加载 ggplot2

下列函数来自 ggplot2：

```
1 ggplot()
2 aes()
3 geom_point()
```

如果文档中没有加载该包，干净的渲染 session 会出现类似：

```
could not find function "ggplot"
```

因此应在文档中明确写出：

```
1 library(palmerpenguins)
2 library(dplyr)
3 library(ggplot2)
```

代码在 Console 中能运行，不代表它在 Quarto 中一定能运行。Console 可能已经加载过 tidyverse，而 Quarto 渲染通常在新的 R session 中从头执行。文档必须自己加载所需的包，不能依赖交互式环境中的历史状态。

5.4 错误三：echo=TREU

错误的 chunk header：

```
```{r, echo=TREU}
summary(penguins)
```
```

TREU 不是 R 的逻辑值。可以改为：

```
```{r, echo=TRUE}
summary(penguins)
```
```

更推荐使用 Quarto 的 YAML 风格 chunk option:

```
```{r}
#| echo: true

summary(penguins)
```
```

echo: true 表示运行代码并在最终文档中显示源代码; echo: false 表示运行代码但隐藏源代码。它本身不决定是否运行这个 chunk。

该错误发生在 knitr 解析 chunk option 的阶段, 甚至可能早于 summary(penguins) 的实际执行。

5.5 完整修复后的 broken.qmd

```
---
title: "Penguin bill measurements"
format: html
---

## First chunk

```{r}
library(palmerpenguins)
library(dplyr)
library(ggplot2)

head(penguins)
```

## Second chunk

```{r}
penguins |>
 filter(species == "Adelie") |>
 ggplot(aes(
 x = bill_length_mm,
 y = bill_depth_mm
)) +
 geom_point()
```

## Third chunk

```{r}
#| echo: true
```

```
summary(penguins)
'''
```

在文件所在目录运行：

```
quarto render broken.qmd
```

成功后会生成 HTML 文件。

5.6 判断错误属于哪一层

报错线索	可能的问题层
Error in filter(), object not found、could not find function	R 代码
Quitting from ... [chunk-name]	knitr 执行 chunk 时停止；根本原因可能仍是 R 错误
错误的 echo、eval 等 chunk option	knitr
YAMLErrorException 或 front matter 格式错误	YAML/Quarto
Pandoc conversion 或 extension 相关错误	Quarto/Pandoc
LaTeX Error、缺少 .sty 文件	LaTeX/PDF 编译

给 chunk 添加名称可以提高可定位性：

```
```{r}
#| label: plot-adelie

...
```
```

此时报错会显示 [plot-adelie]，比 [unnamed-chunk-2] 更容易判断位置。

5.7 HTML 与 PDF 渲染的差别

HTML 渲染大致经过：

```
.qmd -> knitr runs R -> Pandoc -> HTML
```

PDF 渲染还需要 LaTeX：

```
.qmd -> knitr runs R -> Pandoc -> LaTeX -> PDF
```

因此，把 format: html 改为 format: pdf 后出现 No TeX installation was detected，并不说明 R 代码有错，而是 PDF 编译环境缺失。本题的核心是：

找到第一个真正的错误，判断它来自哪一层，修复后重新渲染；不要只盯着最后一行 Execution halted。

## 第二部分 Deutsche Fassung

### 6 Exercise 1: Spaltenargumente mit tidy evaluation

#### 6.1 Problem

```

1 mean_mass_by <- function(data, group_var) {
2 data |>
3 group_by(group_var) |>
4 summarise(
5 mean_mass = mean(body_mass_g, na.rm = TRUE)
6)
7 }

```

`group_by(group_var)` verwendet das Argument nicht automatisch als die beim Funktionsaufruf übergebene Spalte. `dplyr` arbeitet hier mit data masking und tidy evaluation.

#### 6.2 Korrektur

```

1 mean_mass_by <- function(data, group_var) {
2 data |>
3 group_by({{ group_var }}) |>
4 summarise(
5 mean_mass = mean(body_mass_g, na.rm = TRUE),
6 .groups = "drop"
7)
8 }
9
10 mean_mass_by(penguins, species)

```

`{{ group_var }}` übernimmt den nicht als String übergebenen Spaltenausdruck aus dem Funktionsaufruf. Für einen Spaltennamen als String würde man stattdessen `.data[[group_var]]` verwenden.

### 7 Exercise 2: Fehler in einer Hilfsfunktion

#### 7.1 Fehlerursache

```

1 mean_body_mass <- function(x) {
2 mean(x, na.rm = TRUE)
3 }

```

`TREU` ist kein logischer Wert in R. R sucht deshalb nach einem Objekt mit diesem Namen und meldet:

```
object 'TREU' not found
```

Der Fehler wird möglicherweise zunächst von `summarise()` gemeldet, entsteht aber tatsächlich im inneren Aufruf von `mean()`.

## 7.2 Korrektur

```
1 mean_body_mass <- function(x) {
2 mean(x, na.rm = TRUE)
3 }
4
5 summarise_species <- function(data) {
6 data |>
7 group_by(species) |>
8 summarise(
9 mean_mass = mean_body_mass(body_mass_g),
10 .groups = "drop"
11)
12 }
```

Bei verschachtelten Fehlermeldungen sollte deshalb insbesondere der Abschnitt `Caused by error in ...` gelesen werden.

## 8 Exercise 3: Rekursion mit `browser()` untersuchen

### 8.1 Warum die Funktion fehlschlägt

```
1 my_factorial <- function(n) {
2 if (n == 1) return(1)
3 return(n * my_factorial(n - 1))
4 }
```

Für `my_factorial(5)` erreicht die Rekursion den Abbruchfall `n == 1`. Für `my_factorial(0)` entsteht dagegen:

```
0 -> -1 -> -2 -> -3 -> ...
```

Für `my_factorial(3.5)` gilt entsprechend:

```
3.5 -> 2.5 -> 1.5 -> 0.5 -> -0.5 -> ...
```

In beiden Fällen wird 1 niemals erreicht.

### 8.2 Interaktives Debugging

```
1 my_factorial <- function(n) {
2 browser()
3 if (n == 1) return(1)
4 return(n * my_factorial(n - 1))
5 }
```

`Browse[1]>` bedeutet, dass R die Ausführung angehalten hat. Wichtig ist der Namenskonflikt:

```
1 print(n) # Wert der Variablen anzeigen
2 n # Debugger-Befehl "next"
```

| Befehl             | Bedeutung                                             |
|--------------------|-------------------------------------------------------|
| <code>n</code>     | Nächste Anweisung ausführen.                          |
| <code>s</code>     | In einen Funktionsaufruf hineingehen.                 |
| <code>f</code>     | Aktuelle Funktion möglichst bis zum Ende ausführen.   |
| <code>c</code>     | Bis zum nächsten Haltepunkt oder Fehler weiterlaufen. |
| <code>where</code> | Den aktuellen Aufrufstapel anzeigen.                  |
| <code>Q</code>     | Das Debugging sofort abbrechen.                       |

Mehrere Einträge bei `where` zeigen mehrere noch nicht abgeschlossene rekursive Aufrufe. Die äußeren Aufrufe warten jeweils auf das Ergebnis des inneren Aufrufs. Die Zahl in `Browse[1]` ist weder der Wert von `n` noch die Rekursionstiefe.

### 8.3 Korrektur

```
1 my_factorial <- function(n) {
2 stopifnot(
3 n >= 0,
4 n == as.integer(n)
5)
6
7 if (n <= 1) return(1)
8
9 return(n * my_factorial(n - 1))
10 }
```

`stopifnot()` lehnt negative Zahlen und Dezimalzahlen ab. `n <= 1` behandelt sowohl `0!` als auch `1!` korrekt. Nach dem Debugging muss `browser()` entfernt werden.

## 9 Exercise 4: `debug()` und `debugonce()`

### 9.1 Fehleranalyse

```
1 standardise <- function(x) {
2 (x - mean(x)) / sd(x)
3 }
4
5 debugonce(standardise)
```



```
6 standardise(c(1, 2, NA, 4, 5))
```

Im Debugger ergibt sich:

```
1 mean(x) # NA
2 sd(x) # NA
```

Damit wird faktisch  $(x - NA) / NA$  berechnet. Durch die Weitergabe fehlender Werte werden alle Ergebnisse zu NA.

## 9.2 Korrektur

```
1 standardise <- function(x) {
2 (x - mean(x, na.rm = TRUE)) /
3 sd(x, na.rm = TRUE)
4 }
```

Der ursprünglich fehlende Wert bleibt NA. Er wird lediglich bei der Berechnung von Mittelwert und Standardabweichung ignoriert.

| Werkzeug     | Verhalten                            | Beenden                          |
|--------------|--------------------------------------|----------------------------------|
| browser()    | Haltepunkt direkt im Funktionskörper | Zeile aus der Funktion entfernen |
| debugonce(f) | Nur der nächste Aufruf wird debuggt  | automatische Aufhebung           |
| debug(f)     | Jeder weitere Aufruf wird debuggt    | undebbug(f)                      |

# 10 Exercise 5: Ein Quarto-Dokument debuggen

## 10.1 Die drei Fehler

1. In `filter(species = "Adelie")` wurde eine Argumentzuweisung statt eines Vergleichs verwendet. Richtig ist:

```
1 filter(species == "Adelie")
```

2. `ggplot()`, `aes()` und `geom_point()` stammen aus `ggplot2`. Das Paket muss im Dokument explizit geladen werden:

```
1 library(ggplot2)
```

3. `echo=TREU` enthält erneut den Tippfehler `TREU`. Empfohlen ist die Quarto-Schreibweise:

```

'''{r}
#| echo: true
summary(penguins)
'''

```

## 10.2 Korrigiertes Dokument

```

title: "Penguin bill measurements"
format: html

'''{r}
library(palmerpenguins)
library(dplyr)
library(ggplot2)
'''

'''{r}
penguins |>
 filter(species == "Adelie") |>
 ggplot(aes(
 x = bill_length_mm,
 y = bill_depth_mm
)) +
 geom_point()
'''

'''{r}
#| echo: true
summary(penguins)
'''

```

Gerendert wird mit:

```
quarto render broken.qmd
```

## 10.3 Fehlerebenen

| Hinweis                             | Wahrscheinliche Ebene   |
|-------------------------------------|-------------------------|
| Error in filter(), object not found | R                       |
| Quitting from ... [chunk-name]      | knitr stoppt beim Chunk |

| Hinweis                          | Wahrscheinliche Ebene |
|----------------------------------|-----------------------|
| Fehlerhafte Chunk-Option         | knitr                 |
| YAML-Fehler                      | YAML/Quarto           |
| Konvertierungsfehler             | Quarto/Pandoc         |
| LaTeX Error, fehlende .sty-Datei | LaTeX                 |

Quarto führt Chunks von oben nach unten aus. Deshalb wird zunächst nur der erste Fehler sichtbar. Dieser wird korrigiert, anschließend wird erneut gerendert. Ein PDF-Render benötigt zusätzlich eine LaTeX-Installation; ein fehlendes TeX-System ist kein R-Fehler.

## Kurzfazit

- Bei `dplyr`-Funktionen muss die Auswertung von Spaltenargumenten bewusst behandelt werden.
- Verschachtelte Fehlermeldungen müssen bis zur inneren Ursache gelesen werden.
- `browser()`, `debugonce()` und `debug()` halten Funktionen auf unterschiedliche Weise an, verwenden aber dieselben Debugger-Befehle.
- Rekursive Funktionen benötigen einen erreichbaren Abbruchfall und eine Prüfung ihres Eingabebereichs.
- Beim Quarto-Rendering muss zwischen R, knitr, Quarto/Pandoc und LaTeX unterschieden werden.